为Claude构建技能的完整指南

Claude

目录

引言 3

基础知识 4

规划与设计 7

测试与迭代 14

分发与共享 18

模式与故障排除 21

资源与参考 28

介绍

技能是一组指令——以简单的文件夹形式打包——教导Claude如何处理特定任务或工作流程。技能是定制Claude以满足您特定需求的最强大方式之一。通过技能，您可以一次性教会Claude您的偏好、流程和领域专业知识，而无需在每次对话中重新解释。

当您有可重复的工作流程时，技能非常强大：从规格生成前端设计、使用一致的方法进行研究、创建遵循团队风格指南的文档，或协调多步骤流程。它们与Claude的内置功能（如代码执行和文档创建）配合良好。对于构建MCP集成的用户，技能增加了另一个强大的层次，帮助将原始工具访问转化为可靠、优化的工作流程。

本指南涵盖了构建有效技能所需了解的所有内容——从规划和结构到测试和分发。无论您是为自己、团队还是社区构建技能，您都会在其中找到实用的模式和真实的案例。

你将学到的内容：

- 技能结构的技术要求和最佳实践
- 独立技能和MCP增强工作流程的模式
- 我们在不同用例中看到的有效模式
- 如何测试、迭代和分发你的技能

适合谁：

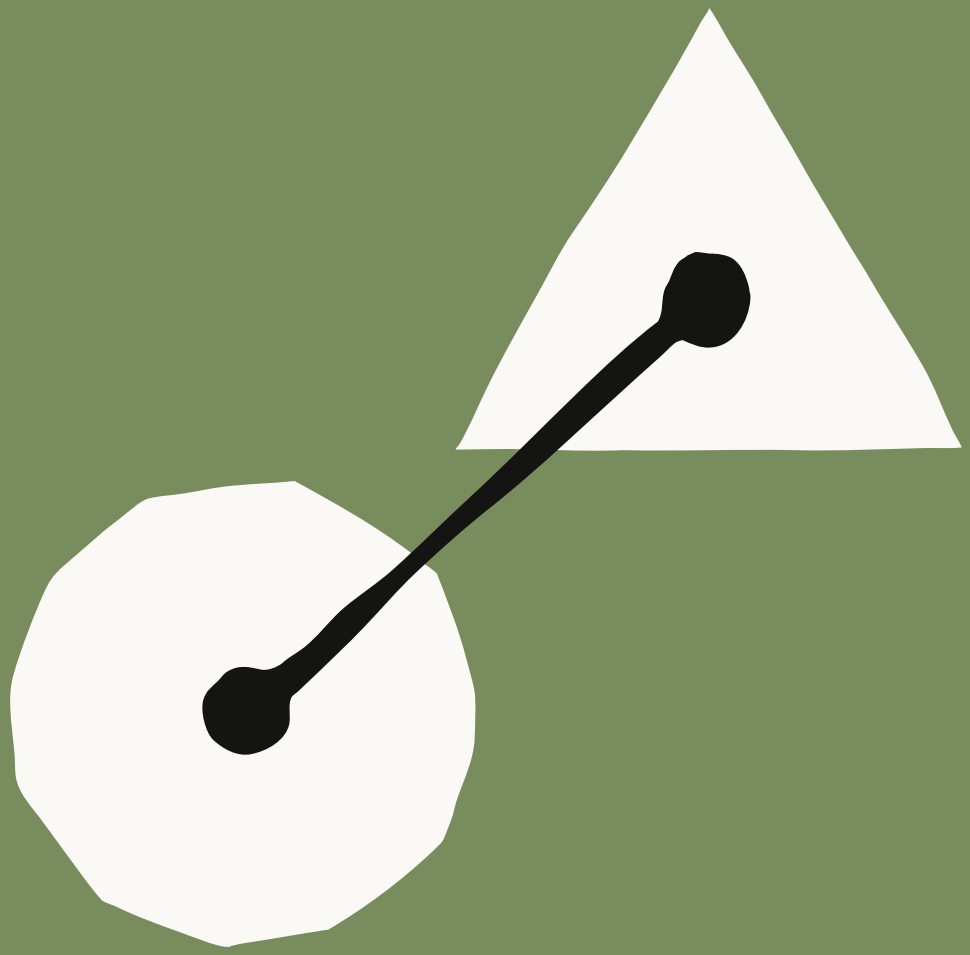
- 希望Claude始终遵循特定工作流程的开发者
- 希望Claude遵循特定工作流程的高级用户
- 希望在组织内标准化Claude工作方式的团队

通过本指南的两条路径

构建独立技能？专注于基础知识、规划和设计，以及类别 1-2。增强 MCP 集成？“技能 + MCP”部分和类别 3 适合你。两条路径共享相同的技术要求，但你可以选择与用例相关的内容。

你将从本指南中获得的内容：到最后，你将能够在一次会议中构建一个功能性技能。预计大约需要 15-30 分钟来使用技能创建器构建和测试你的第一个工作技能。

让我们开始吧。



第一章

基础知识

第一章

基础知识

什么是技能?

技能是一个包含以下内容的文件夹:

- SKILL.md（必需）：包含 YAML 前言的 Markdown 指令
- scripts/（可选）：可执行代码（Python、Bash 等）
- references/ (optional): 根据需要加载的文档
- assets/ (可选): 输出中使用的模板、字体、图标

核心设计原则

渐进式披露

技能使用三级系统:

- 第一层级 (YAML 前言): 始终加载在 Claude 的系统提示中。提供足够的信息，让 Claude 知道何时应该使用每个技能，而无需将所有信息加载到上下文中。
- 第二层级 (SKILL.md 正文): 当 Claude 认为该技能与当前任务相关时加载。包含完整的指令和指导。
- 第三层级 (链接文件): 在技能目录中捆绑的附加文件，Claude 可以根据需要选择导航和发现。

这种渐进式披露在保持专业知识的同时，最小化了令牌的使用。

可组合性

Claude可以同时加载多个技能。你的技能应该能够与其他技能良好协作，而不是假设它是唯一可用的能力。

可移植性

技能在Claude.ai、Claude Code和API上表现一致。只需创建一次技能，它就可以在所有平台上无须修改地使用，前提是环境支持技能所需的任何依赖项。

对于MCP构建者：技能 + 连接器

如果你在没有MCP的情况下构建独立技能？跳到规划和设计 - 你可以随时返回这里。

如果你已经有一个正常工作的MCP服务器，那么你已经完成了艰难的部分。技能是其上的知识层 - 捕捉你已经知道的工作流程和最佳实践，以便Claude能够一致地应用它们。

厨房类比

MCP提供了专业厨房：访问工具、原料和设备。

技能提供了食谱：逐步说明如何创造有价值的东西。

它们共同使用户能够完成复杂的任务，而无需自己逐步弄清楚每个步骤。

它们如何协同工作：

MCP（连接性）	技能（知识）
将Claude连接到您的服务（Notion、Asana、Linear等）	教会Claude如何有效地使用您的服务
提供实时数据访问和工具调用	捕捉工作流程和最佳实践
Claude可以做什么	Claude应该如何做

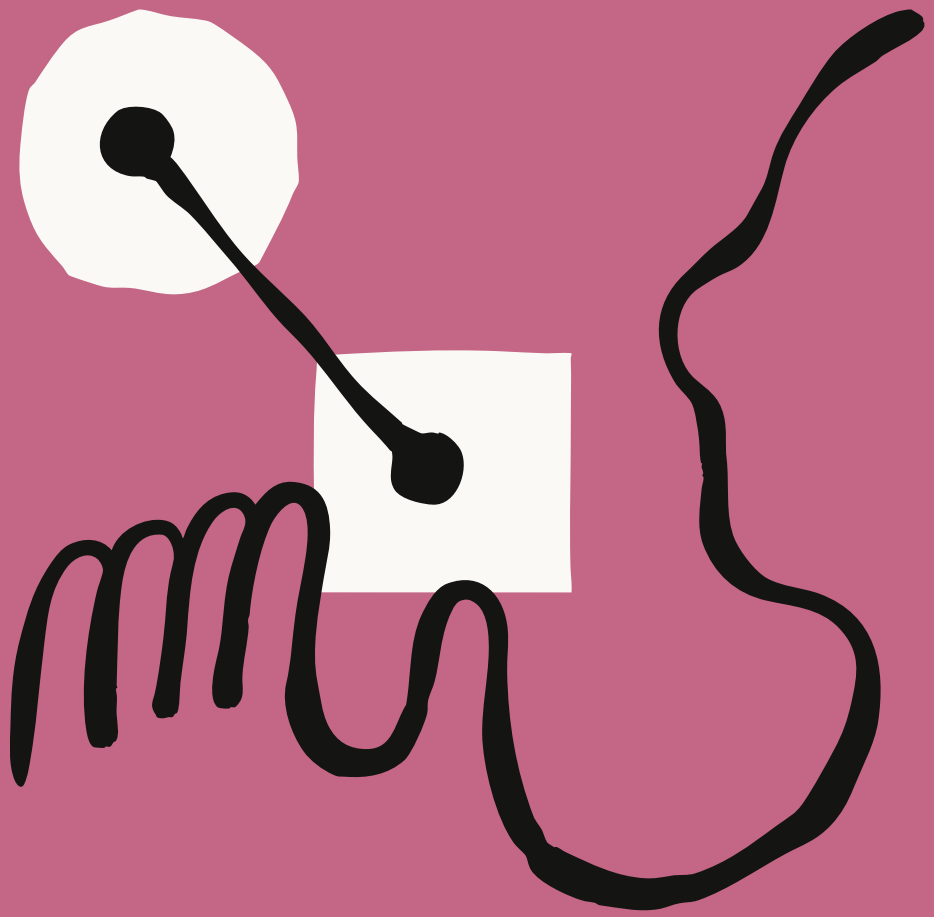
这对您的MCP用户为何重要

没有技能：

- 用户连接您的MCP，但不知道接下来该做什么
- 支持票据询问“我如何使用您的集成做X”
- 每次对话都是从头开始
- 由于用户每次的提示不同，导致结果不一致
- 用户指责您的连接器，而真正的问题在于工作流程指导

有技能：

- 预构建的工作流程在需要时自动激活
- 一致、可靠的工具使用
- 最佳实践嵌入每次互动中
- 更低的集成学习曲线



第2章

规划与设计

第2章

规划与设计

从用例开始

在编写任何代码之前，确定2-3个您的技能应该支持的具体用例。

良好的用例定义：

用例：项目冲刺规划触发条件：用户说“帮我规划这个冲刺”或“创建冲刺任务”

步骤：
1. 从Linear（通过MCP）获取当前项目状态
2. 分析团队的速度和能力
3. 建议任务优先级
4. 在Linear中创建带有适当标签和估算的任务
结果：完全规划的冲刺，任务已创建

问自己：

- 用户想要完成什么？
- 这需要哪些多步骤的工作流程？
- 需要哪些工具（内置的还是MCP？）
- 应该嵌入哪些领域知识或最佳实践？

常见技能使用案例类别

在Anthropic，我们观察到三种常见的使用案例：

类别1：文档和资产创建

用于：创建一致的高质量输出，包括文档、演示文稿、应用程序、设计、代码等。

真实示例： 前端设计技能（另见 docx、pptx、xlsx 和 ppt 的技能）

“创建独特的、生产级的前端界面，具有高设计质量。在构建网页组件、页面、文档、海报或应用程序时使用。”

关键技术：

- 嵌入式风格指南和品牌标准
- 一致输出的模板结构
- 最终确定前的质量检查清单
- 无需外部工具 - 使用 Claude 的内置功能

类别 2：工作流自动化

用于：受益于一致方法的多步骤流程，
包括跨多个 MCP 服务器的协调。

真实示例： 技能创建者技能

“创建新技能的互动指南。引导用户进行用例定义、前言生成、指令编写和验证。”

关键技术：

- 逐步工作流程与验证门
- 常见结构的模板
- 内置审查和改进建议
- 迭代优化循环

类别 3：MCP 增强

用于：工作流程指导，以增强工具访问 MCP 服务器所提供的功能。

真实示例： 哨兵代码审查技能（来自 Sentry）“自动分析和修复在 GitHub Pull Requests 中检测到的错误，使用 Sentry 的错误监控数据通过他们的 MCP 服务器。”

关键技术：

- 按顺序协调多个 MCP 调用
- 嵌入领域专业知识
- 提供用户本来需要指定的上下文
- 常见MCP问题的错误处理

定义成功标准

你如何知道你的技能是否有效？

这些是理想目标 - 粗略的基准而非精确的阈值。追求严谨，但要接受会有基于感觉的评估元素。我们正在积极开发更为稳健的测量指导和工具。

定量指标：

- 技能在90%的相关查询中触发
  - *如何测量：* 运行10-20个应该触发你技能的测试查询。跟踪它自动加载的次数与需要明确调用的次数。
- 在X个工具调用中完成工作流程
  - *如何测量：* 比较启用技能与未启用技能的相同任务。计算工具调用次数和消耗的总令牌数。
- 每个工作流0个失败的API调用
  - *如何测量：* 在测试运行期间监控MCP服务器日志。跟踪重试率和错误代码。

定性指标：

- 用户无需提示Claude关于下一步的操作
  - *如何评估：* 在测试期间，注意你需要重定向或澄清的频率。向测试用户征求反馈。
- 工作流在没有用户修正的情况下完成
  - *如何评估：* 运行相同请求3-5次。比较输出的结构一致性和质量。
- 会话间结果一致
  - *如何评估：* 新用户能否在第一次尝试时在最少指导下完成任务？

技术要求

文件结构

```
你的技能名称/
├── SKILL.md          # 必需 - 主要技能文件
├── scripts/          # 可选 - 可执行代码
│   ├── process_data.py # 示例
│   └── validate.sh # 示例
├── references/       # 可选 - 文档
│   ├── api-guide.md # 示例
│   └── examples/ # 示例
└── assets/           # 可选 - 模板等
    └── report-template.md # 示例
```

关键规则

SKILL.md 命名：

- 必须完全为 `SKILL.md` （区分大小写）
- 不接受变体（SKILL.MD、skill.md 等）

技能文件夹命名：

- **使用短横线命名法：** `notion-project-setup`
- **不允许空格：** `Notion Project Setup`
- **不允许下划线：** `notion_project_setup`
- **不允许大写字母：** `NotionProjectSetup`

不允许 README.md：

- 不要在你的技能文件夹中包含 README.md
- 所有文档都放在 SKILL.md 或 references/ 中。
- 注意：通过 GitHub 分发时，您仍然需要一个仓库级别的 README 供人类用户使用 — 请参见分发和共享。

YAML 前言：最重要的部分

YAML 前言是 Claude 决定是否加载您的技能的方式。确保这一点正确。

最小所需格式

```
---
name: your-skill-name
description: 它的功能。当用户询问 [特定短语] 时使用。

---
```

这就是您开始所需的一切。

字段要求

name（必填）：

- 仅使用 kebab-case
- 不使用空格或大写字母
- 应与文件夹名称匹配

描述（必填）：

- 必须同时包含：
  - 技能的功能
  - 何时使用它（触发条件）
- 不超过 1024 个字符
- 不包含 XML 标签（< 或 >）
- 包括用户可能会说的具体任务
- 如相关，请提及文件类型

许可证（可选）：

- 如果将技能开源，请使用
- 常见：MIT，Apache-2.0

兼容性（可选）

- 1-500个字符
- 指示环境要求：例如，预期产品、所需系统软件包、网络访问需求等。

元数据（可选）：

- 任何自定义键值对
- 建议：作者、版本、mcp-server
- *示例：*
  ```yaml
  元数据：
      作者：ProjectHub
      版本：1.0.0 mcp-server：projecthub
  ```

安全限制

前言中禁止：

- XML 尖括号（< >）
- 名称中包含“claude”或“anthropic”的技能（保留）

原因：前言出现在Claude的系统提示中。恶意内容可能会注入指令。

编写有效技能

描述字段

根据Anthropic的工程博客：“这些元数据……提供了足够的信息，让Claude知道何时应该使用每项技能，而无需将所有信息加载到上下文中。”这是渐进式披露的第一层级。

结构：

```
[它的功能] + [何时使用] + [关键能力]
```

良好描述的示例：

```
# 好 - 具体且可操作
描述：分析 Figma 设计文件并生成
开发者交接文档。当用户上传 .fig 文件时使用
文件，要求提供“设计规格”、“组件文档”或
设计到代码的交接

# 好 - 包含触发短语
管理线性项目工作流程，包括冲刺
规划、任务创建和状态跟踪。当用户提到“冲刺”、“线性任务”、“项目规划”
或询问时使用
创建工单。

# 好 - 清晰的价值主张
描述：PayFlow的端到端客户入职工作流程。
处理账户创建、支付设置，以及
订阅管理。当用户说“加入新
客户设置订阅或创建 PayFlow 账户
```

不良描述的示例：

```
# 太模糊
描述：帮助进行项目。

# 缺失的触发器
描述：创建复杂的多页文档系统。


# 太过技术化，没有用户触发
描述：实现具有
层次关系的项目实体模型。
```

编写主要指令

在前言之后，以Markdown格式编写实际指令。

推荐结构：

根据您的技能调整此模板。用您的具体

内容替换括号中的部分。

```
---
名称：your-skill
描述：[--.]
---

# 您的技能名称

-# 指令

--# 第一步: [第一大步骤]
清晰解释发生了什么。
```

```
示例:
```bash
python scripts/fetch_data.py --project-id PROJECT_ID
预期输出: [描述成功的样子]
```
```

(根据需要添加更多步骤)

示例

示例 1: [常见场景]

用户说: "设置一个新的营销活动"

操作:

1. 通过 MCP 获取现有活动
2. 使用提供的参数创建新活动

结果：活动已创建，带有确认链接

（根据需要添加更多示例）

故障排除

错误：[常见错误信息]

原因：[发生的原因]

解决方案：[如何修复]

（根据需要添加更多错误案例）

操作说明最佳实践

要具体且可操作

好的：

```
运行 `python scripts/validate.py --input {filename}` 来检查
数据格式。
如果验证失败，常见问题包括：
- 缺少必填字段（将其添加到CSV中）
- 日期格式无效（使用YYYY-MM-DD）
```

错误：

```
在继续之前验证数据。
```

包含错误处理

```
-# 常见问题

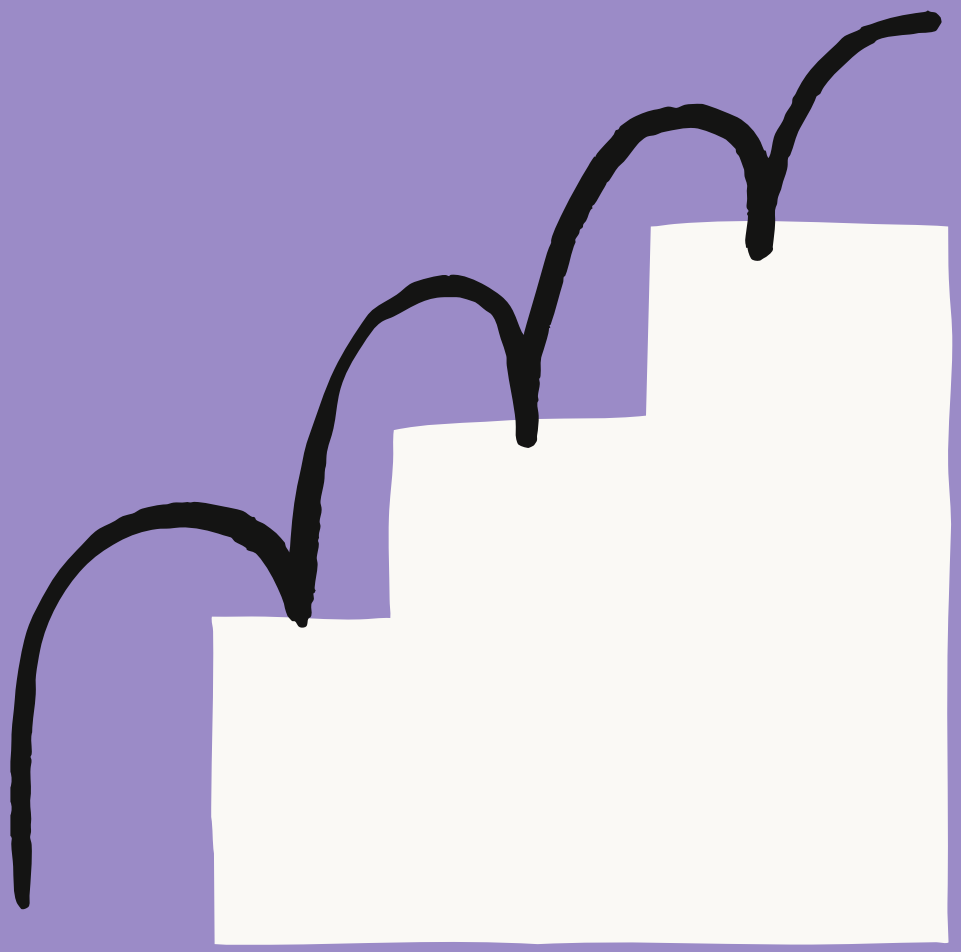
--# MCP连接失败
如果您看到“连接被拒绝”：
1. 验证MCP服务器是否正在运行：检查设置 > 扩展
2. 确认API密钥有效
3. 尝试重新连接：设置 > 扩展 > [您的服务] >
重新连接
```

清晰引用捆绑资源

```
在编写查询之前，请查阅 `references/api-patterns.md`
以获取：
- 速率限制指导
- 分页模式
- 错误代码及处理
```

使用渐进式披露

保持 SKILL.md 专注于核心指令。将详细文档移至 `references/` 并链接到它。（有关三层系统如何工作的，请参见核心设计原则。）



第三章

测试与迭代

第三章

测试与迭代

技能可以根据您的需求在不同的严格程度下进行测试：

- 在Claude.ai中进行手动测试 - 直接运行查询并观察行为。快速迭代，无需设置。
- 在Claude Code中进行脚本测试 - 自动化测试用例，以便在更改中进行可重复的验证。
- 通过技能API进行程序化测试 - 构建评估套件，系统性地针对定义的测试集进行运行。

选择与您的质量要求和技能可见性相匹配的方法。一个由小团队内部使用的技能，其测试需求与部署给数千名企业用户的技能不同。

专业提示：在扩展之前对单一任务进行迭代

我们发现，最有效的技能创建者会在单一具有挑战性的任务上进行迭代，直到Claude成功，然后将成功的方法提取为一个技能。这利用了Claude的上下文学习，并提供了比广泛测试更快的信号。一旦您有了一个有效的基础，就可以扩展到多个测试用例以实现覆盖。

推荐的测试方法

根据早期经验，有效的技能测试通常涵盖三个领域：

1. 触发测试

目标：确保您的技能在正确的时间加载。

测试用例：

- 在明显的任务上触发
- 在改述请求上触发
- 在无关话题上不触发

示例测试套件：

```
应该触发：
- "帮助我设置一个新的ProjectHub工作区"
- "我需要在ProjectHub中创建一个项目"
- "为第四季度规划初始化一个ProjectHub项目"

不应触发：
- "旧金山的天气怎么样？"
- "帮我写Python代码"
- "创建一个电子表格"（除非ProjectHub技能处理
表格）
```

2. 功能测试

目标：验证技能产生正确的输出。

测试用例：

- 生成有效输出
- API 调用成功
- 错误处理正常工作
- 边缘情况已覆盖

示例:

```
测试：创建一个包含5个任务的项目
前提：项目名称为“Q4 规划”，5个任务描述
当：技能执行工作流
那么：
  - 在ProjectHub中创建项目
  - 创建5个具有正确属性的任务
  - 所有任务链接到项目
- 没有API错误
```

3. 性能比较

目标：证明技能提高了与基线的结果。

使用定义成功标准中的指标。以下是比较的样子可能是这样的：

基线比较：

```
没有技能：
- 用户每次提供指令
- 15次来回消息
- 3次失败的API调用需要重试
- 消耗12,000个令牌
```

```
有技能：
- 自动工作流执行
- 仅限2个澄清问题
- 0 次 API 调用失败
- 消耗 6,000 个令牌
```

使用技能创建者技能

技能创建者技能 - 可通过 Claude.ai 的插件目录获取或下载 Claude Code - 可以帮助您构建和迭代技能。如果您有一个 MCP 服务器并且知道您的前 2-3 个工作流程，您可以在一次会议中构建和测试一个功能性技能 - 通常只需 15-30 分钟。

创建技能：

- 从自然语言描述生成技能
- 生成带有前置信息的正确格式的 SKILL.md
- 建议触发短语和结构

审查技能：

- 标记常见问题（模糊描述、缺失触发器、结构问题）
- 识别潜在的过触发/欠触发风险
- 根据技能的声明目的建议测试用例

迭代改进：

- 在使用您的技能并遇到边缘案例或失败后，将这些示例反馈给技能创建者
- 示例：“使用此聊天中识别的问题和解决方案来改进技能如何处理[特定边缘案例]”

使用方法：

> **“使用技能创建者技能来帮助我为[您的使用案例]构建一个技能”**

注意：技能创建者帮助您设计和完善技能，但不执行自动化测试套件或生成定量评估结果。

基于反馈的迭代

技能是活文档。计划根据以下内容进行迭代：

触发不足的信号：

- 技能在应该加载时未加载
- 用户手动启用它
- 关于何时使用它的支持问题

> 解决方案：为描述添加更多细节和细微差别 - 这可能包括特别是技术术语的关键词

触发过度的信号：

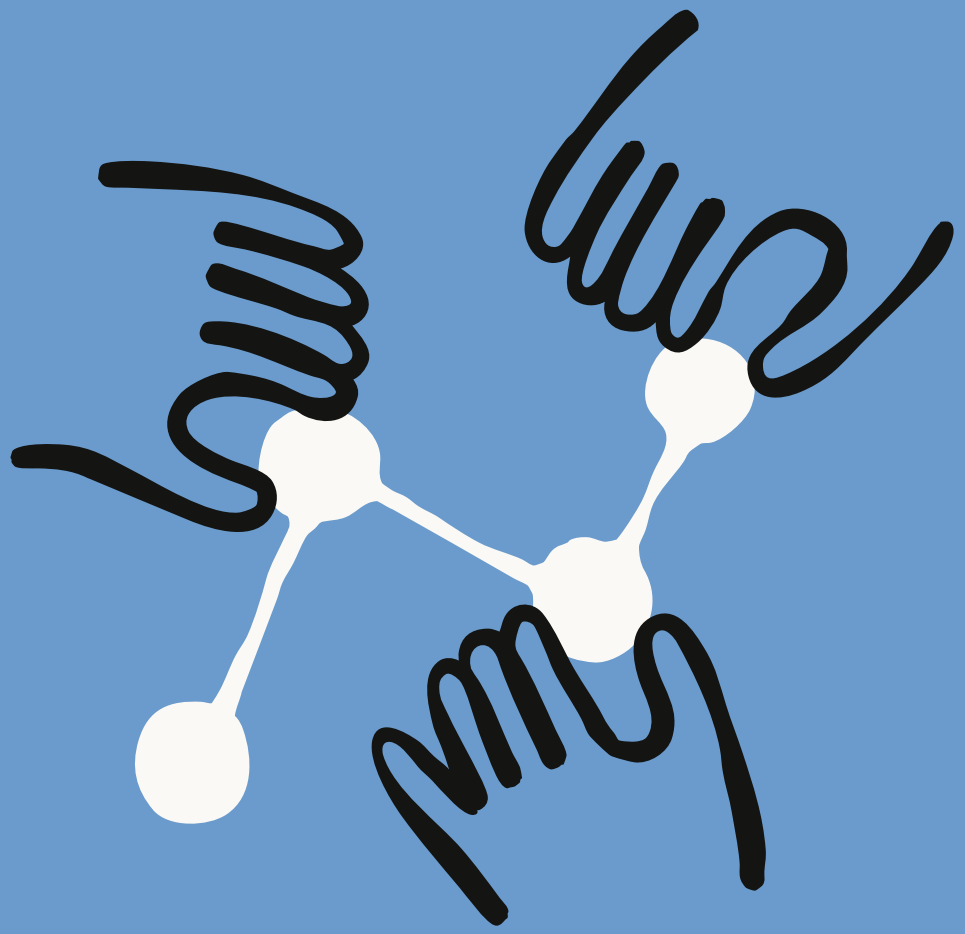
- 技能在不相关的查询中加载
- 用户禁用它
- 对目的的困惑

> 解决方案：添加负触发器，具体化

执行问题：

- 结果不一致
- API调用失败
- 需要用户修正

> 解决方案：改善说明，添加错误处理



第4章

分发和共享

第4章

分发和共享

技能使您的MCP集成更加完整。当用户比较连接器时，具备技能的连接器提供了更快的价值路径，使您在仅使用MCP的替代方案中占据优势。

当前分发模型（2026年1月）

个人用户获取技能的方式：

1. 下载技能文件夹
2. 压缩文件夹（如有需要）
3. 通过设置 > 能力 > 技能上传到Claude.ai
4. 或放置在Claude Code技能目录中

组织级技能：

- 管理员可以在整个工作区部署技能（2025年12月18日发布）
- 自动更新
- 集中管理

开放标准

我们已将代理技能发布为开放标准。与MCP一样，我们认为技能应该能够在工具和平台之间移植——无论您使用Claude还是其他AI平台，同样的技能都应该有效。也就是说，一些技能是为了充分利用特定平台的功能而设计的；作者可以在技能的兼容性字段中注明这一点。我们一直在与生态系统的成员合作制定标准，并对早期的采用感到兴奋。

通过API使用技能

对于程序化用例——例如构建应用程序、代理或利用技能的自动化工作流程——API提供了对技能管理和执行的直接控制。

关键功能：

- `/v1/skills`端点用于列出和管理技能
- 通过`container.skills`参数将技能添加到消息API请求中
- 通过Claude控制台进行版本控制和管理
- 与Claude代理SDK配合使用，以构建自定义代理

何时通过API使用技能与Claude.ai：

用例	最佳表面
最终用户直接与技能互动	Claude.ai / Claude Code
开发过程中的手动测试和迭代	Claude.ai / Claude Code
个体的、临时的工作流程	Claude.ai / Claude Code
以编程方式使用技能的应用程序	API
大规模生产部署	API
自动化管道和代理系统	API

注意：API中的技能需要代码执行工具的测试版，该工具提供了技能运行所需的安全环境。

有关实施细节，请参见：

- 技能 API 快速入门
- 创建自定义技能
- 代理SDK中的技能

今天推荐的方法

首先在 GitHub 上托管您的技能，创建一个公共仓库，清晰的 README（供人类访问者使用——这与您的技能文件夹是分开的，技能文件夹中不应包含 README.md），以及带有截图的示例用法。然后添加一个部分到您的MCP文档，该文档链接到技能，解释了同时使用这两者的价值，并提供了快速入门指南。

1. 在 GitHub 上托管

- 开源技能的公共仓库
- 清晰的README和安装说明
- 示例用法和截图

2. 在您的MCP仓库中记录

- 链接到MCP文档中的技能
- 解释同时使用两者的价值
- 提供快速入门指南

3. 创建安装指南

```
-# 安装[您的服务]技能

1. 下载技能：
- 克隆仓库：`git clone https:-/github.com/yourcompany/
    skills`
- 或从发布页面下载 ZIP 文件

2. 在 Claude 中安装：
- 打开 Claude.ai > 设置 > 技能
  - 点击“上传技能”
  - 选择技能文件夹（压缩文件）

3. 启用技能：
- 切换开启 [您的服务] 技能
- 确保您的 MCP 服务器已连接

4. 测试：
- 询问 Claude：“在 [您的服务] 中设置一个新项目”
```

定位您的技能

您如何描述您的技能决定了用户是否理解其价值并实际尝试使用它。在撰写关于您技能的内容时——无论是在您的自述文件、文档还是营销材料中——请牢记以下原则。

关注结果，而非特性：

好:

```
ProjectHub技能使团队能够建立完整的项目
在几秒钟内创建工作区——包括页面、数据库和
模板——而不是花费30分钟进行手动设置。
```

错误:

```
ProjectHub技能是一个文件夹，包含YAML前言和Markdown指令，用于调用我们的
MCP服务器工具。
```

突出MCP + 技能的故事：

```
我们的MCP服务器为Claude提供访问您的Linear项目的权限。
我们的技能教会克劳德你们团队的冲刺规划工作流程。
它们共同实现了基于人工智能的项目管理。
```

第5章

模式与故障排除

第5章

模式与故障排除

这些模式源于早期采用者和内部团队创造的技能。
它们代表了我们看到的有效的常见方法，而不是规定性的模板。

选择你的方法：以问题为先还是以工具为先

可以把它想象成家得宝。你可能会带着一个问题走进店里——“我需要修理一个厨房橱柜”——然后一名员工指引你找到合适的工具。或者你可能会挑选一个新的电钻，并询问如何将其用于你的特定工作。

技能的工作方式也是如此：

- 以问题为先：“我需要设置一个项目工作区” → 你的技能按正确的顺序协调合适的MCP调用。用户描述结果；技能处理工具。
- 以工具为先：“我已经连接了Notion MCP” → 你的技能教会Claude最佳工作流程和最佳实践。用户可以访问；技能提供专业知识。

大多数技能倾向于某一方向。了解哪种框架适合您的使用案例有助于您选择下面的正确模式。

模式 1：顺序工作流编排

使用场景：您的用户需要按特定顺序进行多步骤流程。

示例结构：

```
-# 工作流：新客户入职

--# 步骤 1：创建账户
调用 MCP 工具：`create_customer`
参数：姓名，电子邮件，公司

--# 步骤 2：设置支付
调用 MCP 工具：`setup_payment_method`
等待：支付方式验证

--# 步骤 3：创建订阅
调用 MCP 工具： `create_subscription`
参数： plan_id, customer_id（来自步骤 1）

--# 步骤 4：发送欢迎邮件
调用 MCP 工具： `send_email`
模板： welcome_email_template
```

关键技术：

- 明确的步骤顺序
- 步骤之间的依赖关系
- 每个阶段的验证
- 失败时的回滚指令

模式 2：多 MCP 协调

在以下情况下使用：工作流程跨越多个服务。

示例：设计到开发的交接

```
--# 阶段 1：设计导出 (Figma MCP)
1. 从 Figma 导出设计资产
2. 生成设计规范
3. 创建资产清单

--# 阶段 2：资产存储 (Drive MCP)
1. 在 Drive 中创建项目文件夹
2. 上传所有资产
3. 生成可分享链接

--# 阶段 3：任务创建 (Linear MCP)
1. 创建开发任务
2. 将资产链接附加到任务
3. 分配给工程团队

--# 第四阶段：通知（Slack MCP）
1. 将交接总结发布到 #engineering
2. 包含资产链接和任务参考
```

关键技术：

- 清晰的阶段划分
- MCP之间的数据传递
- 在进入下一个阶段之前进行验证
- 集中式错误处理

模式 3：迭代优化

使用时：输出质量随着迭代而提高。

示例：报告生成

```
-# 迭代报告创建

--# 初步草稿
通过MCP获取数据
生成初稿报告
保存到临时文件

--# 质量检查
运行验证脚本：`scripts/check_report.py`
识别问题：
    - 缺失部分
- 格式不一致
- 数据验证错误

--# 精炼循环
处理每个识别的问题
2. 重新生成受影响的部分
3. 重新验证
4. 重复直到达到质量阈值

--# 最终化
1. 应用最终格式
2. 生成摘要
3. 保存最终版本
```

关键技术：

- 明确的质量标准
- 迭代改进
- 验证脚本
- 知道何时停止迭代

模式 4：上下文感知工具选择

使用时：相同结果，根据上下文选择不同工具。

示例：文件存储

```
-# 智能文件存储

--# 决策树
1. 检查文件类型和大小
2. 确定最佳存储位置：
- 大文件（>10MB）：使用云存储 MCP
- 协作文档：使用 Notion/Docs MCP
- 代码文件：使用 GitHub MCP
    - 临时文件：使用本地存储

--# 执行存储
基于决策：
- 调用适当的 MCP 工具
- 应用特定服务的元数据
- 生成访问链接

--# 向用户提供上下文
解释为什么选择该存储
```

关键技术：

- 清晰的决策标准
- 备用选项
- 关于选择的透明度

模式 5：特定领域智能

使用时：您的技能提供了超出工具访问的专业知识。

示例：财务合规

```
-# 合规的支付处理

--# 处理前（合规检查）
1. 通过 MCP 获取交易详情
2. 应用合规规则：
    - 检查制裁名单
- 验证管辖权允许事项
    - 评估风险等级
3. 文档合规决策

--# 处理
如果合规通过：
  - 调用支付处理MCP工具
  - 应用适当的欺诈检查
  - 处理交易
否则：
  - 标记以供审核
  - 创建合规案例

--# 审计追踪
- 记录所有合规检查
- 记录处理决策
- 生成审计报告
```

关键技术：

- 领域专业知识嵌入逻辑
- 行动前的合规性
- 综合文档
- 清晰的治理

故障排除

技能无法上传

错误：“无法在上传的文件夹中找到 SKILL.md”

原因：文件未准确命名为 SKILL.md

解决方案：

- 将文件重命名为 SKILL.md（区分大小写）
- 验证方法：ls -la 应显示 SKILL.md

错误：“无效的前置信息”

原因：YAML 格式问题

常见错误：

```
# 错误 - 缺少分隔符
name: my-skill
description: 做事情

# 错误 - 引号未闭合
name: my-skill
description: "做事情

# 正确
---
name: my-skill
描述：执行任务
```

错误: "无效的技能名称"

原因: 名称包含空格或大写字母

```
# 错误
名称: 我的酷技能

# 正确
名称: my-cool-skill
```

技能未触发

症状: 技能从未自动加载

修复:

修订您的描述字段。请参见描述字段以获取好/坏示例。

快速检查清单:

- 这太笼统了吗？（“帮助项目”行不通）
- 它是否包含用户实际会说的触发短语？
- 如果适用，它是否提到了相关的文件类型？

调试方法：

问Claude：“你什么时候会使用[技能名称]技能？”Claude会引用描述。根据缺失的内容进行调整。

技能触发过于频繁

症状：技能在不相关的查询中加载

解决方案：

1. 添加负触发器

```
描述：用于CSV文件的高级数据分析。用于
统计建模、回归、聚类。请勿用于
简单数据探索（请使用数据可视化技能）。
```

2. 更具体一些

```
# 太宽泛
处理文档

# 更具体
处理PDF法律文件以进行合同审查
```

3. 明确范围

```
描述：PayFlow 电子商务支付处理。使用
专门用于在线支付工作流程，而非一般的
财务查询。
```

MCP连接问题

症状：技能加载但MCP调用失败

检查清单：

1. 验证MCP服务器是否已连接
   - Claude.ai: 设置 > 扩展 > [您的服务]
   - 应显示“已连接”状态
2. 检查身份验证
   - API 密钥有效且未过期
   - 已授予适当的权限/范围
   - OAuth 令牌已刷新
3. 独立测试 MCP
   - 要求 Claude 直接调用 MCP（不使用技能）
   - “使用 [服务] MCP 获取我的项目”
   - 如果失败，问题在于 MCP 而非技能
4. 验证工具名称
   - 技能引用正确的MCP工具名称
   - 检查MCP服务器文档
   - 工具名称区分大小写

未遵循指示

症状：技能加载但Claude不遵循指示

常见原因：

1. 指示过于冗长
   - 保持指示简洁
   - 使用项目符号和编号列表
   - 将详细参考移至单独文件
2. 指示被埋没
   - 将重要指示放在顶部
   - 使用 ## 重要 或 ## 关键 标题
   - 如有必要，重复关键点
3. 模糊的语言

```
# 不好
确保正确验证事项

# 好
关键：在调用 create_project 之前，请验证：
- 项目名称不能为空
- 至少分配一个团队成员
- 开始日期不能在过去
```

高级技术：对于关键验证，考虑捆绑一个脚本以编程方式执行检查，而不是依赖语言指令。代码是确定性的；语言解释则不是。请参见Office技能以获取此模式的示例。

4. 模型“懒惰” 添加明确的鼓励：

```
-# 性能说明
- 花时间彻底完成这项工作
- 质量比速度更重要
- 不要跳过验证步骤
```

注意：将此添加到用户提示中比在SKILL.md中更有效

大型上下文问题

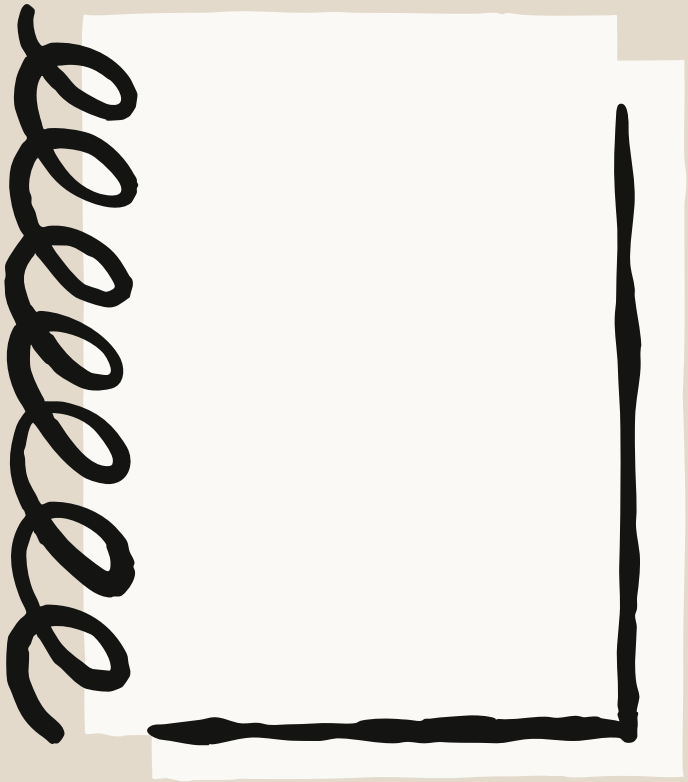
症状：技能似乎很慢或响应下降

原因：

- 技能内容过大
- 同时启用的技能过多
- 加载了所有内容，而不是逐步披露

解决方案：

1. 优化 SKILL.md 大小
   - 将详细文档移至 references/
   - 链接到参考文献，而不是内联
   - 保持 SKILL.md 在 5,000 字以内
2. 减少启用的技能
   - 评估是否同时启用了超过 20 - 50 个技能
   - 推荐选择性启用
   - 考虑与相关能力的“技能包”



第六章

资源和参考文献

第六章

资源和参考文献

如果您正在构建您的第一个技能，请先从最佳实践指南开始，然后根据需要参考API文档。

官方文档

Anthropic资源：

- 最佳实践指南
- 技能文档
- API 参考
- MCP 文档

博客文章：

- 介绍代理技能
- 工程博客：为现实世界装备代理
- 技能解析
- 如何为 Claude 创建技能
- 为 Claude 编写技能代码
- 通过技能改善前端设计

示例技能

公共技能库：

- GitHub: **anthropics/skills**
- 包含可自定义的Anthropic创建的技能

工具和实用程序

技能创建者技能：

- 内置于Claude.ai并可用于Claude Code
- 可以根据描述生成技能
- 审查并提供建议
- 使用："帮助我使用技能创建者构建技能"

验证：

- 技能创建者可以评估您的技能
- 询问："审查此技能并建议改进"

获取支持

技术问题：

- 一般问题：Claude开发者Discord上的社区论坛

报告错误：

- GitHub问题：anthropics/skills/issues
- 包括：技能名称、错误信息、重现步骤

参考A：快速检查清单

使用此检查清单在上传前后验证您的技能。如果您想要更快的开始，请使用技能创建者技能生成您的初稿，然后通过此列表检查确保您没有遗漏任何内容。

在您开始之前

- ☐识别了2-3个具体的使用案例
- ☐识别的工具（内置或MCP）
- ☐审查了本指南和示例技能
- ☐规划了文件夹结构

在开发过程中

- ☐文件夹名称采用kebab-case
- ☐存在SKILL.md文件（准确拼写）
- ☐YAML前置内容有---分隔符
- ☐名称字段：kebab-case，无空格，无大写字母
- ☐描述包括WHAT和WHEN
- ☐没有任何XML标签（< >）
- ☐指令清晰且可操作
- ☐包含错误处理
- ☐提供了示例
- ☐参考文献清晰链接

上传前

- ☐测试了明显任务的触发
- ☐测试了改述请求的触发
- ☐验证了不会在无关主题上触发
- ☐功能测试通过
- ☐工具集成工作（如适用）
- ☐压缩为.zip文件

上传后

- ☐在真实对话中测试
- ☐监控触发不足/过度触发
- ☐ 收集用户反馈
- ☐对描述和说明进行迭代
- ☐在元数据中更新版本

参考B: YAML 前言

必填字段

```
---
名称：技能名称（用连字符分隔）
描述：它的功能和使用时机。包括特定的
触发短语。
---
```

所有可选字段

```
名称：技能名称
描述：[必填描述]
许可证：MIT # 可选：开源许可证
允许的工具："Bash(python:*) Bash(npm:*) WebFetch" # 可选：
限制工具访问
元数据：# 可选：自定义字段
    作者：公司名称
版本：1.0.0
   mcp-server：服务器名称
    类别：生产力
标签：[项目管理，自动化]
 文档：https:-/example.com/docs
 支持：support@example.com
```

安全注意事项

允许：

- 任何标准的 YAML 类型（字符串、数字、布尔值、列表、对象）
- 自定义元数据字段
- 长描述（最多 1024 个字符）

禁止：

- XML 尖括号（< >） - 安全限制
- 在 YAML 中执行代码（使用安全的 YAML 解析）
- 以 "claude" 或 "anthropic" 前缀命名的技能（保留）

参考 C：完整技能示例

要展示本指南中模式的完整、可生产的技能：

- 文档技能 - PDF、DOCX、PPTX、XLSX 创建
- 示例技能 - 各种工作流程模式
- 合作伙伴技能目录 - 查看来自各个合作伙伴的技能，如 Asana，Atlassian，Canva，Figma，Sentry，Zapier 等

这些库保持最新，并包含超出此处所涵盖内容的额外示例。克隆它们，修改以适应您的用例，并将其用作模板。

Claude

claude.ai